

Análise de Redes de Colaboração Open Source com Teoria de Grafos

github.com/h3js/h3

Gabriel Felipe Santos de Sousa · João Lucas Gonçalves Teles
Rodrigo Albuquerque Rocha · Rafa Barbosa · Eron Arthur da Silva

O que vamos apresentar

01

Escolha do Repositório

Por que h3js/h3?

02

Estratégia de Extração

GraphQL · filtros · pesos

03

Como Funciona a Extração

API GitHub · Issues · PRs

04

Classes Auxiliares & Estrutura

Modularização do código

05

Construção dos Grafos

4 grafos com propósitos distintos

06

Exportação para GEPHI

Formato GEXF

Métricas & Resultados

Conclusão

01 · Escolha do Repositório

✗ Primeira Tentativa

hyprwm/Hyprland

- 34.300+ estrelas
- Compositor Wayland em C++
- Grafo gerado com estrutura incorreta
- Issues fechadas sem autor identificável
- Commits diretos sem reviews
- Resultado: grafo esparso sem colaboração real



✓ Repositório Escolhido

h3js/h3

- 5.300+ estrelas (acima do mínimo exigido)
- Framework HTTP minimalista em TypeScript
- Comunidade ativa com reviews e merges frequentes
- Interações ricas entre colaboradores distintos
- Discussões técnicas ativas em issues e PRs
- Grafo denso e representativo da colaboração real

O enunciado exige repositório público com mais de 5.000 estrelas, garantindo comunidade ativa.

02 · Estratégia de Extração

Por que GraphQL e não REST?

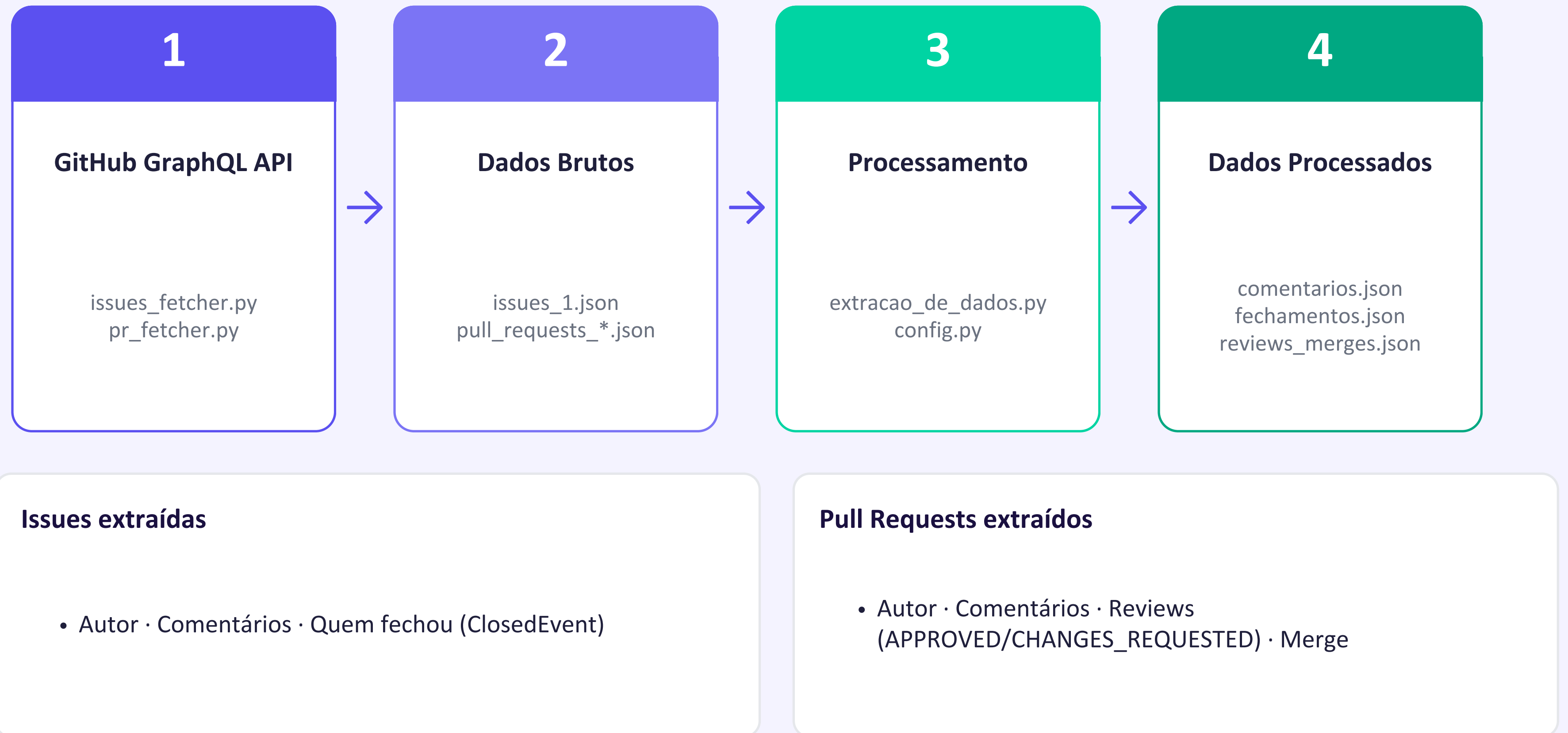
- Uma única requisição busca tudo
- REST: 1 requisição por issue/PR/comentário
- Menos requisições = menor risco de rate limit
- Dados chegam estruturados e prontos

O que coletamos

Tipo de Interação	Peso	Arquivo
Comentário em issue/PR	2	comentarios.json
Fechamento de issue	3	fechamentos.json
Review / Aprovação de PR	4	reviews_merges.json
Merge de PR	5	reviews_merges.json

Regra: auto-interações e usuários sem login são ignorados. Apenas interações entre usuários DISTINTOS viram arestas.

03 · Como Funciona a Extração



04 · Classes Auxiliares & Estrutura de Classes

Modularização

O sistema separa a lógica de comunicação externa (API) da persistência de dados local (JSON), garantindo que a construção do grafo seja independente da fonte de dados.

```
class LeituraJson:  
    def carregar_dados(caminho):  
        with open(caminho, 'r') as f:  
            return json.load(f)  
  
class GitHubClient:  
    def get_data(self, endpoint):  
        # Gerencia autenticação  
        return requests.get(url,...)
```

Hierarquia de Classes

AbstractGraph

Interface abstrata — define operações comuns

AdjacencyListGraph

Otimizado para redes esparsas (comum em OSS)

AdjacencyMatrixGraph

Alternativa para redes densas

Polimorfismo permite calcular métricas independente da estrutura de armazenamento.

05 · Construção dos Grafos

G1

Comentários

Issues e PRs

527

Vértices

853

Arestas

3.802

Peso Total

G2

Fechamentos

de Issues

102

Vértices

100

Arestas

384

Peso Total

G3

Reviews & Merges

Aprovações de PR

158

Vértices

200

Arestas

3.454

Peso Total

G4

Grafo Integrado

Todos os tipos

564

Vértices

978

Arestas

7.640

Peso Total

06 · Exportação para GEPHI

Formato GEXF

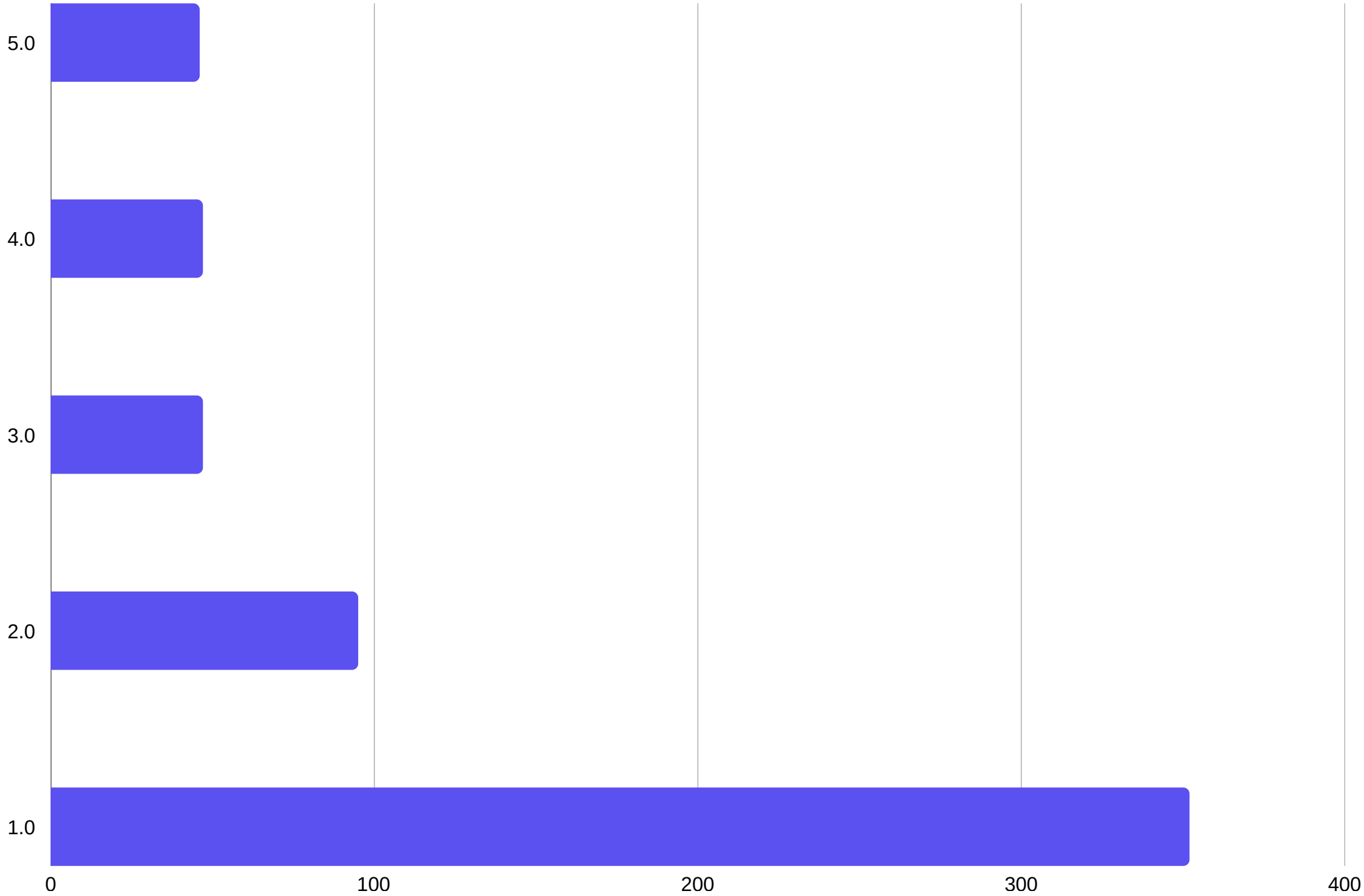
- Graph Exchange XML Format
- Exportado via ExportacaoGexf.py
- Contém vértices com labels (login)
- Arestas com peso das interações
- Direcionado: de → para
- Compatível com GEPHI para visualização

```
<node id="pi0" label="pi0"/>  
<edge source="pi0" target="nozomuikuta"  
weight="5"/>
```

Interface Textual (GUI)

- gui_backend.py + gui_views.py
- Métricas calculadas e exportadas:
 - resumo_grafos.csv
 - top5_centralidade.csv
 - comunidades_detectadas.csv
 - resultado_gui.json
- RodarInterfaceGrafos.py → executa tudo

07 · Métricas — Top 5 Centralidade de Grau



352

pi0

Grau total — hub central do projeto

62.5%

Centralidade

pi0 domina o grafo integrado

5.039

Peso Ponderado

interações pesadas de pi0

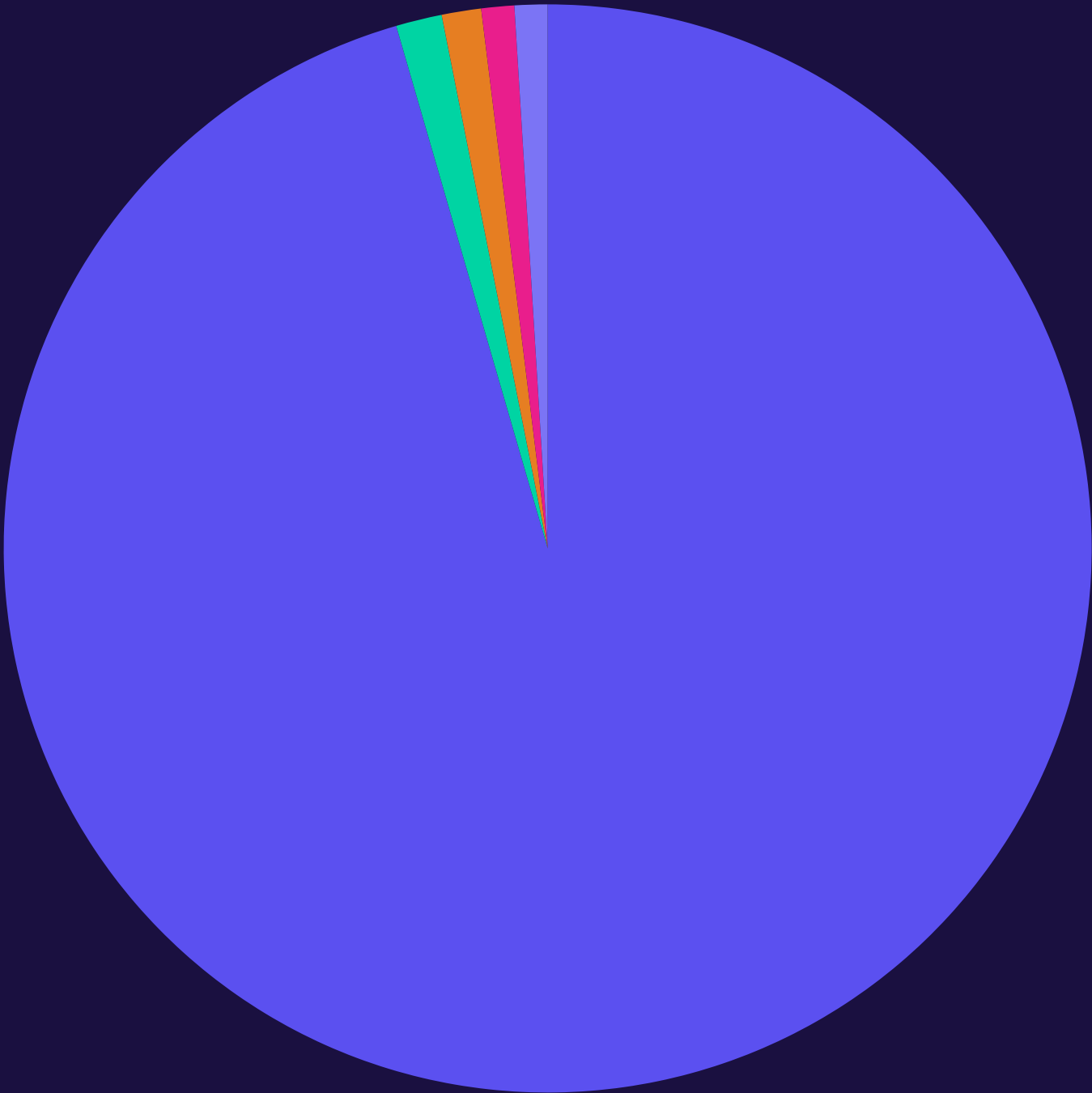
5

Comunidades

detectadas no grafo integrado

07 · Métricas — Comunidades & Densidade

Comunidade 1 (489) Comunidade 2 (7) Comunidade 3 (6)
Comunidade 4 (5) Comunidade 5 (5)



Resumo dos Grafos

Grafo	Vértices	Arestas	Densidade
G1 Comentários	527	853	0,006
G2 Fechamentos	102	100	0,019
G3 Reviews/Merges	158	200	0,016
G4 Integrado	564	978	0,006

Principais Achados

- 1 comunidade gigante (489 membros) dominante
- Densidade baixa — grafo esparso típico de OSS
- G2 tem menor densidade mas maior estrutura

O que aprendemos



Teoria de Grafos aplicada

Redes de colaboração revelam padrões invisíveis na análise textual.



GraphQL é superior ao REST

Para extração em massa de dados do GitHub, GraphQL reduz drasticamente o número de requisições.



Modularização importa

Separar extração, construção e análise facilitou debug, testes e extensão do projeto.



OSS é centralizado

O h3js/h3 tem um único hub (pi0) dominante — padrão esperado em projetos open source.



Lista vs Matriz

Lista de adjacência é ideal para grafos esparsos; ambas chegam ao mesmo resultado.